

Grupo 1. Análisis Estadístico Multivariado

Etapa 1. Conociendo el negocio

Raúl Gómez Muñoz L0XXXXXXX
Blanca Ruiz Hernández L0XXXXXXX

19 de octubre de 2026

Etapa 1. Conociendo el negocio

Tecnológico de Monterrey

Tabla de contenidos

1. Método de mínimos cuadrados 1

1. Método de mínimos cuadrados

Supongamos que tenemos un conjunto de datos

$$((x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n))$$

Por ejemplo, consideremos el siguiente conjunto de datos simulado:

```
library(tidyverse)
library(krulRutils)
library(latex2exp)

set.seed(123)

n <- 50
x_data <- runif(n, min = 0, max = 10)
epsilon_data <- rnorm(n, mean = 0, sd = 1)
y_data <- x_data + epsilon_data

sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que podemos visualizar estos datos mediante un diagrama de dispersión:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "Datos simulados",
    x = TeX("$X_{0}$"),
    y = TeX("$Y_{0}$")
  ) +
  theme_krul()

sample_data_plot
```

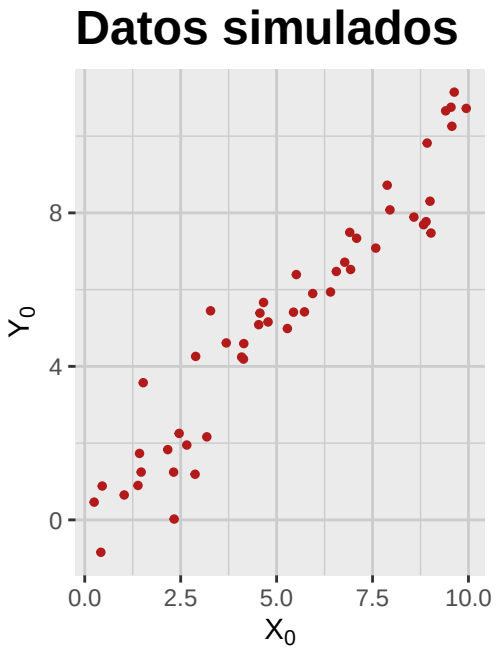


Figura 1: Diagrama de dispersión de los datos simulados.

Nos gustaría encontrar la línea recta que mejor se ajuste a estos datos. Hay varias formas de definir "mejor ajuste", pero una de las más comunes es minimizar la suma de los errores al cuadrado (SSE por sus siglas en inglés) entre los valores observados y los valores correspondientes sobre la línea recta propuesta. De manera más precisa, buscamos una recta

$$Y_0 = \hat{f}(X_0) = \hat{\beta}_0 + \hat{\beta}_1 X_0$$

tal que si definimos $\hat{y}_i = \hat{f}(x_i)$ entonces minimicemos la suma

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{1}$$

```
fitted_data_tbl <- sample_data_tbl %>%
  mutate(
    fitted = fitted(lm(y ~ x, data = .))
  )

fitted_data_plot <- fitted_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  geom_segment(
    aes(xend = x, yend = fitted),
    color = c_pal("C green"),
    alpha = 1
  ) +
```

```
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +
  geom_point(
    aes(x = x, y = fitted),
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "Recta ajustada",
    x = TeX("$X_{0}$"),
    y = TeX("$Y_{0}$")
  ) +
  theme_krul()

fitted_data_plot
```

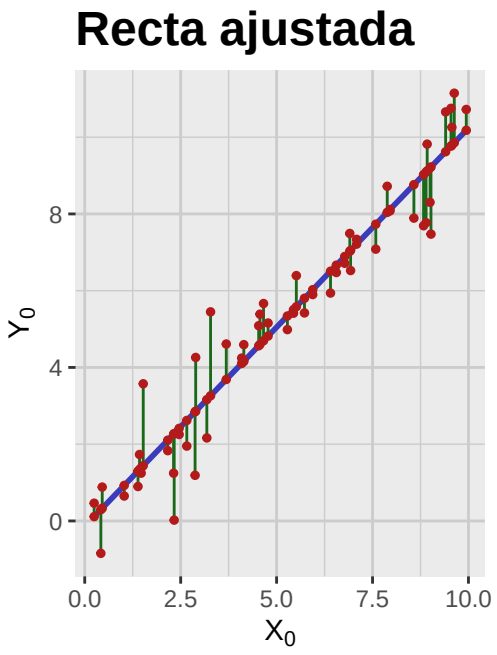


Figura 2: Línea recta que mejor se ajusta a los datos simulados, bajo el criterio de mínimos cuadrados. En otras palabras, esta es la línea recta que minimiza el SSE.

Para encontrar la fórmula para $\hat{\beta}_0$ y $\hat{\beta}_1$ en términos de nuestros datos, utilizaremos un poco de

álgebra lineal. Empecemos por fijar los vectores

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \qquad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

que llamaremos el *vector de las observaciones independientes* y el *vector de las observaciones dependientes*, respectivamente. Fijamos ahora la llamada *matriz de diseño*, dada por,

$$\Phi = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

y notamos que si definimos

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

entonces $\hat{y} = \Phi\hat{\beta}$ donde

$$\hat{\beta} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix}$$

Notemos que para minimizar la suma de los errores al cuadrado dada en la ecuación (1), necesitamos encontrar la proyección ortogonal del vector y sobre el espacio generado por las columnas de la matriz Φ. Empezamos por calcular la *matriz de coefficientes*

$$D^2 = \left(\Phi^T\Phi\right)^{-1} = \begin{bmatrix} \frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} & -\frac{\bar{x}}{S_{xx}} \\ -\frac{\bar{x}}{S_{xx}} & \frac{1}{S_{xx}} \end{bmatrix}$$

donde

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{y} \qquad S_{xx} = \sum_{i=1}^{n}(x_i - \bar{x})^2$$

Calculamos ahora la *matriz de estimadores de mínimos cuadrados*

$$A = D^2\Phi^T = \left(\Phi^T\Phi\right)^{-1}\Phi^T = \begin{bmatrix} \frac{1}{n} + \frac{\bar{x}(\bar{x}-x_1)}{S_{xx}} & \cdots & \frac{1}{n} + \frac{\bar{x}(\bar{x}-x_n)}{S_{xx}} \\ \frac{x_1-\bar{x}}{S_{xx}} & \cdots & \frac{x_n-\bar{x}}{S_{xx}} \end{bmatrix}$$

y, finalmente, calculamos la *matriz sombrero*

$$H = \Phi A = \Phi\left(\Phi^T\Phi\right)^{-1}\Phi^T$$

Notemos que las entradas de la matriz H están dadas por:

$$h_{ij} = \frac{1}{n} + \frac{(x_i - \bar{x})(x_j - \bar{x})}{S_{xx}}$$

en particular,

$$h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{S_{xx}}$$

Utilizando estas matrices, podemos calcular

$$\hat{\beta} = Ay \qquad \text{y} \qquad \hat{y} = Hy$$

Para ilustrar este procedimiento, consideremos el siguiente ejemplo.

Ejemplo. Consideremos el siguiente conjunto de datos:

```
set.seed(123)

n <- 5
x_data <- 1:n
epsilon_data <- rnorm(n, mean = 0, sd = n / 10)
y_data <- x_data + epsilon_data
sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que los vectores de observaciones independientes y dependientes son:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \qquad y = \begin{bmatrix} 0.72 \\ 1.88 \\ 3.78 \\ 4.04 \\ 5.06 \end{bmatrix}$$

y la matriz de diseño es:

```
Phi <- matrix(c(rep(1, n), x_data), nrow = n, ncol = 2)
```

$$\Phi = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}$$

Calculamos ahora la matriz de coeficientes:

```
D2 <- solve(t(Phi) %*% Phi)
```

$$D^2 = \begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix}$$

y la matriz de estimadores de mínimos cuadrados:

```
A <- D2 %*% t(Phi)
```

$$A = \begin{bmatrix} 0.8 & 0.5 & 0.2 & -0.1 & -0.4 \\ -0.2 & -0.1 & 0 & 0.1 & 0.2 \end{bmatrix}$$

Finalmente, la matriz sombrero es:

```
H <- Phi %*% A
```

$$H = \begin{bmatrix} 0.6 & 0.4 & 0.2 & 0 & -0.2 \\ 0.4 & 0.3 & 0.2 & 0.1 & 0 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0 & 0.1 & 0.2 & 0.3 & 0.4 \\ -0.2 & 0 & 0.2 & 0.4 & 0.6 \end{bmatrix}$$

Utilizando estas matrices, podemos rápidamente calcular los vectores $\hat{\beta}$ y $\hat{y}$:

```
beta_hat <- A %*% y_data
y_hat <- H %*% y_data
```

$$\hat{\beta} = \begin{bmatrix} -0.16 \\ 1.08 \end{bmatrix} \qquad \hat{y} = \begin{bmatrix} 0.93 \\ 2.01 \\ 3.1 \\ 4.18 \\ 5.26 \end{bmatrix}$$

Notemos que en este caso la suma de los errores al cuadrado (SSE) es:

```
sse <- sum((y_data - y_hat)^2)
```

$$\text{SSE} = 0.59$$

La siguiente figura ilustra el ajuste por mínimos cuadrados para este conjunto de datos:

```
fitted_example_data_tbl <- sample_data_tbl %>%
  mutate(
    fitted = as.vector(y_hat)
  )
fitted_example_data_plot <- fitted_example_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  geom_segment(
    aes(xend = x, yend = fitted),
    color = c_pal("C green"),
    alpha = 1
  ) +
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +
```

```
  geom_point(
    aes(x = x, y = fitted),
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "Recta ajustada",
    x = TeX("$X_{0}$"),
    y = TeX("$Y_{0}$")
  ) +
  theme_krul()
fitted_example_data_plot
```

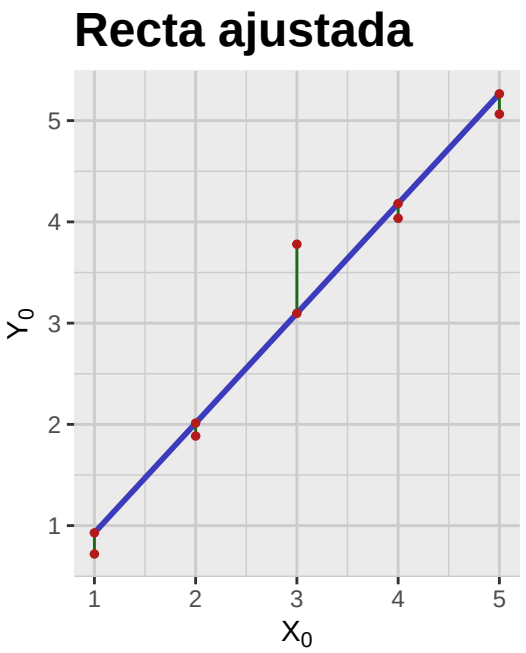


Figura 3: Ajuste por mínimos cuadrados para el conjunto de datos del ejemplo.

Notemos que este proceso se puede aplicar a cualquier conjunto de datos dado, sin embargo una de las aplicaciones más comunes de este método es tratar de predecir el valor de la variable Y_0 dado un valor específico de la variable X_0 que no se encuentre en el conjunto de datos original. Se sigue que para que estas predicciones sean de utilidad, es necesario que el comportamiento de nuestros datos siga una tendencia lineal, ya que en caso contrario las predicciones podrían no ser confiables. Por ejemplo, consideremos el ajuste por el método de mínimos cuadrados para los siguientes conjuntos de datos:

```r
library(patchwork)

set.seed(123)

n <- 500
x_data1 <- runif(n, min = 0, max = 10)
epsilon_data1 <- rnorm(n, mean = 0, sd = 50)
y_data1 <- (x_data1)^3 + epsilon_data1

x_data2 <- runif(n, min = 0, max = 10)
epsilon_data2 <- rnorm(n, mean = 0, sd = 0.1)
y_data2 <- sin(x_data2) + epsilon_data2

x_data3 <- runif(n, min = 0, max = 10)
y_data3 <- runif(n, min = 0, max = 10)

x_data4 <- runif(n, min = 0, max = 10)
y_data4 <- mapply(function(mu, sigma) rnorm(1, mean = mu, sd = sigma),
                  mu = x_data4,
                  sigma = x_data4)

sample_data1_tbl <- tibble(
  x = x_data1,
  y = y_data1
)

sample_data2_tbl <- tibble(
  x = x_data2,
  y = y_data2
)

sample_data3_tbl <- tibble(
  x = x_data3,
  y = y_data3
)

sample_data4_tbl <- tibble(
  x = x_data4,
  y = y_data4
)

sample_data1_plot <- sample_data1_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
```

```
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos cúbicos",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data2_plot <- sample_data2_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos senoidales",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data3_plot <- sample_data3_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos aleatorios",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data4_plot <- sample_data4_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
```

```
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos con varianza creciente",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data_plot <- (sample_data1_plot + sample_data2_plot) /
  (sample_data3_plot + sample_data4_plot)

sample_data_plot
```

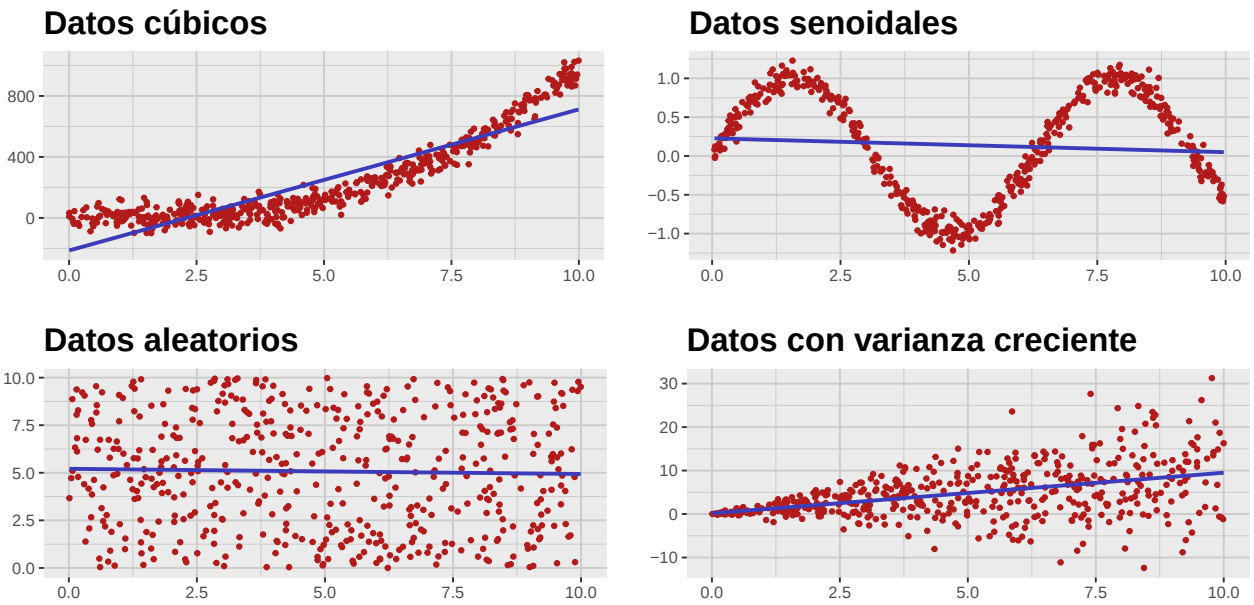


Figura 4: Ajuste por mínimos cuadrados para diferentes conjuntos de datos.

Es bastante claro que en estos casos el ajuste por mínimos cuadrados no es el método adecuado para realizar predicciones confiables. En general, este método solo es efectivo cuando se cumplen ciertas hipótesis que estudiaremos en las siguientes secciones, cuando introduciremos el modelo de regresión lineal desde un punto de vista estadístico.